L'ÉDITION — Frontend Author Authentication, Registration & Concurrent Refresh Guide

> **Target API:** `MagizineAuthor.Api` (`http://localhost:5267` | `https://localhost:7137`)
>
> **Architecture:** Author Self-Registration (`IsApproved` Editorial Gate) + Short-Lived JWT `AccessToken` (`120m`) + Rotating `RefreshToken` (`7d`) + `HttpOnly` Cookies (`author_access_token` / `author_refresh_token`) + **Frontend Mutex Queue** + **Backend 30-Second Grace Period**

1. End-to-End Author Authentication & Concurrent Refresh Flow

```
sequenceDiagram
    autonumber
    actor Browser as Author Portal (Multi-Tabs / Parallel Requests)
    participant API as MagizineAuthor.Api
    participant Svc as AuthService
    participant DB as SQL Server (Tbl_Author)

    Note over Browser,DB: 1. Optional Self-Registration (POST /api/author/auth/Register)
    Browser->>API: POST /api/author/auth/Register { FirstName, LastName, Email, Password, ... }
    API->>DB: INSERT Tbl_Author (IsApproved = 0, IsActive = 1)
    API-->>Browser: 200 OK (Pending editorial approval by Admin)

    Note over Browser,DB: 2. Approved Author Login (POST /api/author/auth/Login)
    Browser->>API: POST /api/author/auth/Login { EmailOrSlug, Password }
    API->>Svc: Verify BCrypt PasswordHash + Enforce IsApproved == 1 & IsActive == 1
    Svc->>DB: Save SHA256(Token 1) -> RefreshTokenHash, Set LastLoginAt = UtcNow
    API-->>Browser: Set-Cookie: author_access_token (HttpOnly, Path=/)\nSet-Cookie: author_refresh_token (Htt

    Note over Browser: 3. Requests A, B, and C fail simultaneously with 401
    Browser->>Browser: Acquire Mutex lock (isRefreshing = true); push Requests B & C into failedQueue
    Browser->>API: Request A calls POST /api/author/auth/Refresh (Sends Token 1 Cookie)
    API->>Svc: RefreshTokenAsync(Token 1)
    Svc->>DB: Move Token 1 -> PreviousRefreshTokenHash (Grace Period: UtcNow + 30s)\nSave SHA256(Token 2) ->
    API-->>Browser: Set-Cookie (AccessToken + Token 2) + 200 OK (IsGracePeriodHit: false)
    Browser->>Browser: Release Mutex lock; retry queued Requests B & C

    rect rgb(255, 240, 240)
    Note over Browser,DB: 4. Fallback: Another browser tab triggers Refresh with Token 1 within 30s
    Browser->>API: Tab 2 calls POST /api/author/auth/Refresh (Still carrying Token 1)
    API->>Svc: Verify SHA256(Token 1) == PreviousRefreshTokenHash & PreviousRefreshTokenExpiresAt > UtcNow
    API-->>Browser: Set-Cookie (AccessToken, preserving Token 2 in Cookie Jar) + 200 OK (IsGracePeriodHit: tr
    end
```

2. Cookie Isolation & Token Specification

To allow an Editor/Admin and an Author to be signed in simultaneously in the same browser on `localhost`, `MagizineAuthor.Api` uses dedicated cookie names:

Cookie / Token	Name	Format	Lifetime	Flags	Path Scope
Access Token Cookie	`author_access_token`	Signed HMAC-SHA256 JWT (`typ: "access"` , `role: "Author"`)	`120 minutes`	`HttpOnly` , `Secure` , `SameSite=Lax`	`/` (Sent on all Author API calls)
Refresh Token Cookie	`author_refresh_token`	64-byte CSPRNG (`Base64Url`)	`7 days`	`HttpOnly` , `Secure` , `SameSite=Strict`	`/api/author/auth` (Sent only to `/Refresh` & `/Logout`)
Grace Period Record	`PreviousRefreshTokenHash`	SHA-256 hex digest in `dbo.Tbl_Author`	`30 seconds`	Server-side DB column	Prevents multi-tab race conditions

---

3. Author Auth Endpoints Reference (`/api/author/auth`)

3.1 `POST /api/author/auth/Register` (`[AllowAnonymous]`)

Submits a new Author registration. The account is created with `IsApproved = false` and `IsActive = true` until an Admin approves the author via `POST /api/admin/author/ChangeStatus` .

Request Body:

```
{
  "FirstName": "Julian",
  "LastName": "Vance",
  "Email": "julian.vance@ledition.com",
  "Password": "Author@123",
  "ConfirmPassword": "Author@123",
  "Title": "Architectural Critic",
  "Bio": "Writes on brutalist geometry and Nordic interiors.",
  "PhotoUrl": null,
  "InstagramUrl": "https://instagram.com/julianvance",
  "TwitterUrl": null,
  "WebsiteUrl": "https://julianvance.com"
}
```

3.2 `POST /api/author/auth/Login` (`[AllowAnonymous]`)

Authenticates an approved Author by `Email` (`elena.rostova@ledition.com`) or `Slug` (`elena-rostova`).

- Returns `ME#401` if credentials are invalid.
- Returns `ME#403` if `IsApproved == false` (`"Your author account is pending editorial approval by an administrator."`) or `IsActive == false` .

Request Body:

```
{
  "EmailOrSlug": "elena.rostova@ledition.com",
  "Password": "Admin@123"
}
```

Response (`200 OK`):

- Sets `HttpOnly` cookies `author_access_token` and `author_refresh_token` .


```
{
  "IsSuccess": true,
  "IsError": false,
  "RespType": "Success",
  "RespCode": "MS#000",
  "RespDesp": "Signed in successfully.",
  "Data": {
    "AccessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "TokenType": "Bearer",
    "ExpiresAt": "2026-10-11T13:00:00+00:00",
    "RefreshTokenExpiresAt": "2026-10-18T11:00:00+00:00",
    "SessionId": "01J...",
    "IsGracePeriodHit": false,
    "Author": {
      "AuthorId": "01JAUTHOR0000000000000001",
      "FirstName": "Elena",
      "LastName": "Rostova",
      "Email": "elena.rostova@ledition.com",
      "Title": "Senior Editor & Cultural Essayist",
      "Bio": "Elena Rostova writes on quiet luxury, minimalist aesthetics, and modern literature.",
      "PhotoUrl": "https://images.unsplash.com/photo-1534528741775-53994a69daeb",
      "InstagramUrl": "https://instagram.com/elenarostova",
      "TwitterUrl": "https://twitter.com/elenarostova",
      "WebsiteUrl": "https://elenarostova.com",
      "Slug": "elena-rostova",
      "IsApproved": true,
      "IsActive": true,
      "LastLoginAt": "2026-10-11T11:00:00+00:00",
      "CreatedAt": "2026-10-10T00:00:00+00:00",
      "UpdatedAt": "2026-10-11T11:00:00+00:00"
    }
  }
}
```

3.3 `POST /api/author/auth/Refresh` (`[AllowAnonymous]`)

Rotates the session using the `author_refresh_token` `HttpOnly` cookie (with a 30-second grace period on `PreviousRefreshTokenHash`).

```
{}
```

3.4 `GET /api/author/auth/Me` (or `POST /api/author/auth/Me`) (`[Authorize]`)

Returns the authenticated Author's `AuthorResModel` profile.

3.5 `POST /api/author/auth/UpdateProfile` (`[Authorize]`)

Updates the authenticated Author's profile details (`FirstName`, `LastName`, `Email`, `Title`, `Bio`, `PhotoUrl`, `InstagramUrl`, `TwitterUrl`, `WebsiteUrl`).

3.6 `POST /api/author/auth/ChangePassword` (`[Authorize]`)

Changes the Author's password, revokes `RefreshTokenHash` & `PreviousRefreshTokenHash`, and clears the `HttpOnly` cookies.

```
{
  "CurrentPassword": "Admin@123",
  "NewPassword": "Author@1234",
  "ConfirmNewPassword": "Author@1234"
}
```

3.7 `POST /api/author/auth/Logout` (`[AllowAnonymous]`)

Clears `RefreshTokenHash`, `RefreshTokenExpiresAt`, `PreviousRefreshTokenHash`, and `PreviousRefreshTokenExpiresAt` in `dbo.Tbl_Author` and deletes both `HttpOnly` cookies.

```
{}
```

---

4. Production Frontend Mutex Queue Interceptor for Author Portal (Axios /

TypeScript)

```
import axios, { AxiosError, InternalAxiosRequestConfig } from 'axios';

interface RetryableRequestConfig extends InternalAxiosRequestConfig {
  _retry?: boolean;
}

interface QueuedRequest {
  resolve: () => void;
  reject: (error: unknown) => void;
}

export const authorApi = axios.create({
  baseURL: import.meta.env.VITE_AUTHOR_API_URL ?? 'http://localhost:5267',
  withCredentials: true, // CRITICAL: Sends & receives author_access_token / author_refresh_token HttpOnly co
  headers: {
    'Content-Type': 'application/json',
  },
});

let isRefreshing = false;
let failedQueue: QueuedRequest[] = [];

const processQueue = (error: unknown = null) => {
  failedQueue.forEach((prom) => {
    if (error) {
      prom.reject(error);
    } else {
      prom.resolve();
    }
  });
  failedQueue = [];
};

authorApi.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const originalRequest = error.config as RetryableRequestConfig | undefined;

    if (!originalRequest || error.response?.status !== 401) {
      return Promise.reject(error);
    }

    if (
      originalRequest.url?.includes('/api/author/auth/Login') ||
      originalRequest.url?.includes('/api/author/auth/Register') ||
      originalRequest.url?.includes('/api/author/auth/Refresh')
    ) {
      return Promise.reject(error);
    }

    if (originalRequest._retry) {
      return Promise.reject(error);
    }

    if (isRefreshing) {
      return new Promise<void>((resolve, reject) => {
        failedQueue.push({ resolve, reject });
      }).then(() => {
        originalRequest._retry = true;
```

```
        return authorApi(originalRequest);
      });
    }

    originalRequest._retry = true;
    isRefreshing = true;

    try {
      const refreshRes = await authorApi.post('/api/author/auth/Refresh', {});

      if (!refreshRes.data?.IsSuccess) {
        throw new Error(refreshRes.data?.RespDesp || 'Author session expired');
      }

      processQueue(null);
      return authorApi(originalRequest);
    } catch (refreshError) {
      processQueue(refreshError);
      window.location.href = '/author/login';
      return Promise.reject(refreshError);
    } finally {
      isRefreshing = false;
    }
  }
);
```